Credit Risk: Loan Default Predictor

Interview Preparation — 40 Questions & Answers

Topic	Questions
Data & Leakage	Q1 – Q10
Modeling & Machine Learning	Q11 – Q25
MLOps & Production	Q26 – Q35
Ethics & Regulation	Q36 – Q40

Section 1: Data & Leakage

Q1. What is data leakage and why is it critical to avoid in credit risk modeling?

Data leakage occurs when information that would not be available at prediction time is accidentally included during model training, causing unrealistically high performance that collapses in production. In credit risk, post-origination columns such as total payments received, recovery amounts, or late fee charges are recorded after the loan outcome is already known. Including them lets the model 'see the future', making it appear highly accurate in testing but completely useless for predicting new loan defaults.

Q2. What are examples of post-origination leakage columns in LendingClub data?

Examples include: total_pymnt (total amount paid by borrower), recoveries (post charge-off recovery amount), collection_recovery_fee, last_pymnt_amnt, out_prncp (outstanding principal), and total_rec_int. These columns are populated only after the loan has matured or defaulted, so they directly encode the outcome and must be removed before training.

Q3. Why did you use a time-based train/test split instead of a random split?

Loans are not independent across time — economic conditions, lending standards, and borrower behaviour change over years. A random split would allow the model to train on 2018 data and test on 2015 data, leaking future trends into the past. A time-based split (e.g., train on 2007–2016, validate on 2017, test on 2018) mimics how the model would be deployed in production and gives an honest estimate of generalisation performance.

Q4. Why were joint-application secondary-applicant fields dropped?

These fields (e.g., sec_app_fico_range_low, sec_app_open_acc) had more than 95% missing values because the vast majority of LendingClub loans are individual applications. Imputing or retaining them would introduce noise and could bias the model, so they were removed during the cleaning step.

Q5. How did you handle missing values in the dataset?

The approach depends on the feature type. For numeric features with low missingness (< 5%), median imputation is appropriate. For high-missingness features, they are either dropped or a binary missingness indicator is added. Categorical features may use a dedicated 'Unknown' category. Importantly, imputation statistics (medians, modes) are computed only on the training set and then applied to the validation and test sets to prevent leakage.

Q6. What is the difference between data/raw, data/interim, and data/processed?

data/raw contains the original compressed CSV files from LendingClub — these are never modified. data/interim holds intermediate outputs such as feature-engineered datasets before final cleaning. data/processed contains the final cleaned parquet file (loans_cleaned.parquet) used directly for model training. This separation makes the pipeline reproducible and auditable.

Q7. Why do you use Parquet instead of CSV for the processed dataset?

Parquet is a columnar binary format that offers significant advantages: roughly 10x smaller file size due to compression, much faster read speeds (especially column-selective queries), native support for data types (so integers and floats are not re-parsed as strings), and compatibility with pandas, Spark, and Arrow ecosystems. For a 1.37M row dataset, this difference is substantial.

Q8. How did you reduce memory usage when loading the feature matrix?

By casting float features from float64 to float32, memory usage is halved with negligible loss in precision for ML purposes. Integer columns can similarly be downcasted. Using pyarrow to read parquet also avoids loading the full schema at once. For very large datasets, chunked reading or Dask can be used.

Q9. How did you decide which features to keep from the original 84 columns?

Feature selection followed several steps: (1) remove obvious leakage columns, (2) drop near-zero variance features, (3) remove features with excessive missingness (> 50–70%), (4) analyse correlation to remove redundant features, (5) use domain knowledge (e.g., FICO score, DTI, annual income are well-known credit risk drivers), and (6) validate feature importance using the trained model's built-in or SHAP-based scores.

Q10. What is the Pandas 3.x Copy-on-Write change and why does it matter?

In Pandas 3.x, Copy-on-Write (CoW) is the default behaviour, meaning that any operation on a DataFrame slice returns a new object rather than modifying in place. Using inplace=True on chained assignments or modifying a slice silently may no longer work as expected or raises warnings. The correct pattern is to reassign: df = df.fillna(value) or df['col'] = df['col'].fillna(value), ensuring predictable, bug-free transformations.

Section 2: Modeling & Machine Learning

Q11. Why is predicting loan default a classification problem, not regression?

The target variable — whether a loan defaults — is a binary categorical outcome (1 = default, 0 = no default). Regression predicts a continuous quantity (like a probability score directly), while classification models predict class membership and can output calibrated probabilities. Using classification framing allows direct optimisation of metrics like AUC-ROC and F1, and enables threshold-based decision rules aligned with business policy.

Q12. How did you define the target variable 'default'?

The target is 1 for loans labelled 'Charged Off', 'Default', or 'Late (31-120 days)', and 0 for 'Fully Paid'. Loans with statuses like 'Current' or 'In Grace Period' are excluded from training since their final outcome is unknown — including them would introduce noise into the target label.

Q13. The default rate is ~21.2%. Is this imbalanced? How do you handle it?

A 21% minority class is moderately imbalanced but not extreme (unlike fraud detection at < 0.1%). Strategies include: (1) class_weight='balanced' in sklearn to up-weight minority samples, (2) adjusting

the decision threshold below 0.5, (3) oversampling with SMOTE on the training set only, (4) using evaluation metrics that are robust to imbalance such as AUC-PR (precision-recall curve) and F1 rather than accuracy.

Q14. Why is accuracy a misleading metric for this problem?

With a 21.2% default rate, a model that always predicts 'no default' achieves 78.8% accuracy while being completely useless. Metrics that account for class imbalance — such as AUC-ROC, F1, precision, recall, and the area under the precision-recall curve — provide a much more honest assessment of model performance and business utility.

Q15. What is AUC-ROC and why is it useful here?

AUC-ROC (Area Under the Receiver Operating Characteristic curve) measures the model's ability to rank defaulters above non-defaulters across all possible decision thresholds. An AUC of 1.0 is perfect; 0.5 is random. It is threshold-agnostic and insensitive to class imbalance, making it a good overall measure of discriminative ability for loan default prediction.

Q16. What is the precision-recall tradeoff in credit risk context?

Precision = of all loans predicted as default, how many actually defaulted. Recall = of all actual defaults, how many did the model catch. A high-recall model captures most defaulters but may deny too many good loans (false positives = lost revenue). A high-precision model is selective but misses risky loans (false negatives = credit losses). The business cost of each error type determines where to set the threshold.

Q17. Why might you choose XGBoost or LightGBM over logistic regression?

XGBoost and LightGBM are gradient boosted tree ensembles that automatically capture non-linear feature interactions, handle missing values natively, and are robust to outliers and different feature scales. Logistic regression is simpler and more interpretable but assumes linear decision boundaries, which is unlikely to hold in complex credit data. Tree models typically achieve 5–15% higher AUC on structured tabular data.

Q18. What is cross-validation and how would you apply it to this dataset?

Cross-validation estimates model generalisation by training and evaluating on multiple data folds. For time-series data like loans, TimeSeriesSplit is used instead of standard k-fold to respect temporal ordering. The dataset is split into k sequential folds; training always uses earlier data to predict later data. This prevents future information leaking into training and gives a realistic performance estimate.

Q19. How would you tune hyperparameters for XGBoost or LightGBM?

Common approaches: (1) Grid Search — exhaustive but expensive; (2) Random Search — samples parameter combinations randomly, often finds good solutions faster; (3) Bayesian Optimisation with Optuna or Hyperopt — builds a probabilistic model of the objective to target promising regions. Key hyperparameters include learning_rate, n_estimators, max_depth, subsample, colsample_bytree, and reg_alpha/reg_lambda. All tuning should be done on the validation set, with the test set held out until the very end.

Q20. What is overfitting and how did you prevent it?

Overfitting occurs when a model memorises training data patterns rather than learning generalisable rules, resulting in high training performance but poor test performance. Prevention strategies include: regularisation (L1/L2 penalties, tree depth limits), early stopping (stop training when validation loss stops improving), cross-validation, and using a held-out test set for final evaluation only.

Q21. How do you choose a decision threshold for your classifier?

The default threshold of 0.5 is rarely optimal. The best threshold balances the business cost of false positives (denying good loans) against false negatives (approving bad loans). Using the precision-recall curve or an ROC curve, you identify the threshold that maximises the F1 score, or you directly optimise an expected profit/loss function. In regulated lending, the threshold may also be constrained by approval rate targets.

Q22. What is SMOTE and when would you use it?

SMOTE (Synthetic Minority Over-sampling Technique) creates synthetic minority-class samples by interpolating between existing minority samples in feature space. It is useful when the class imbalance is severe (< 5% minority). For this project's 21% default rate, SMOTE is likely unnecessary and class_weight balancing or threshold tuning is preferred. SMOTE must only be applied to the training set to avoid data leakage into the validation/test sets.

Q23. What features do you expect to be the most predictive of loan default?

Based on domain knowledge and typical LendingClub analysis: FICO range (credit score), debt-to-income ratio (dti), loan grade (assigned by LendingClub), interest rate (int_rate), annual income, loan amount, employment length, number of delinquencies, revolving balance utilisation, and loan purpose. SHAP analysis on the trained model can confirm and rank these quantitatively.

Q24. How do you interpret a confusion matrix for this problem?

A confusion matrix shows: True Positives (correctly predicted defaults), True Negatives (correctly predicted non-defaults), False Positives (good loans incorrectly flagged as default — opportunity cost), and False Negatives (actual defaults missed — credit loss). In credit risk, False Negatives are typically more costly because they represent loans that default and cause real financial loss to the lender.

Q25. How would you handle categorical features like loan_purpose or home_ownership?

Low-cardinality categoricals (e.g., home_ownership with 5 values) can be one-hot encoded. Higher-cardinality features (e.g., employer title) require target encoding, frequency encoding, or grouping rare categories into an 'Other' bin. Tree-based models like LightGBM can handle categorical features natively without encoding. All encodings must be fitted on training data only.

Section 3: MLOps & Production

Q26. How does your FastAPI inference service work?

The FastAPI service exposes a POST endpoint (e.g., /predict) that accepts a JSON payload containing loan application features. Internally, the service loads a serialised model (e.g., pickled XGBoost or MLflow model artifact), runs the same preprocessing pipeline applied during training, and returns a predicted default probability and a binary decision. FastAPI is chosen for its async performance, automatic OpenAPI docs, and Pydantic v2 request validation.

Q27. What is MLflow and why did you use it for experiment tracking?

MLflow is an open-source platform for managing the ML lifecycle. It logs: parameters (hyperparameters used), metrics (AUC, F1 at each epoch), artifacts (model files, feature importance plots, confusion matrices), and code version. This makes experiments fully reproducible — any team member can re-run or compare any past experiment using the MLflow UI. The project stores all runs in the mlruns/ directory.

Q28. What would you monitor in production for a loan default model?

Key monitoring dimensions: (1) Data drift — check whether input feature distributions shift over time using PSI (Population Stability Index) or KL divergence; (2) Concept drift — monitor whether the relationship between features and default changes (e.g., during economic downturns); (3) Model performance — track AUC, precision, recall as ground truth labels arrive; (4) Prediction distribution — alert if the model's output probability distribution shifts significantly; (5) Latency and throughput for API health.

Q29. What is model drift and how would you detect it?

Model drift occurs when the statistical properties of input data or the target relationship change over time, degrading model performance. Data drift (covariate shift) is detected by comparing feature distributions between training data and recent production data using PSI or statistical tests (KS test, chi-squared). Concept drift requires monitoring the model's actual prediction accuracy against ground truth outcomes as they become available.

Q30. How would you handle model versioning in production?

MLflow Model Registry provides model versioning with stages (Staging, Production, Archived). When a new model is trained and validated, it is registered in MLflow, promoted to Staging for A/B testing, and then promoted to Production only if it outperforms the current model on held-out data. The FastAPI service loads the 'Production' stage model, making rollback as simple as changing the stage tag.

Q31. What is the difference between batch and real-time inference?

Real-time (online) inference processes one loan application at a time with low latency (milliseconds) — suitable for web-based loan applications where an immediate decision is needed. Batch inference processes thousands of existing loans overnight — suitable for portfolio risk reviews or pre-scoring marketing lists. This project's FastAPI service supports real-time inference; batch scoring can be added as a separate pipeline using pandas and the same model.

Q32. Why do you use pathlib.Path instead of string paths?

pathlib.Path provides an object-oriented, cross-platform API for file paths that is safer and more readable than string concatenation. Paths derived from __file__ using Path(__file__).parent.parent ensure the code works regardless of the working directory from which it is invoked, which is critical for production services and scheduled pipelines running in different environments.

Q33. How do you ensure reproducibility of your experiments?

Reproducibility is ensured by: (1) setting random seeds in numpy, sklearn, and the model library; (2) logging all hyperparameters to MLflow; (3) pinning dependency versions in requirements.txt or pyproject.toml; (4) using the same train/test split logic with a fixed seed or time-based split; (5) committing config YAML files to git rather than hardcoding values in scripts.

Q34. What is the purpose of configs/ YAML files in the project?

YAML config files externalise model hyperparameters and feature lists from the code, making experimentation easier and cleaner. Changing a hyperparameter does not require modifying source code — only updating the YAML and re-running the training script. This also makes config changes reviewable in git, which is valuable for auditing model versions in a regulated environment.

Q35. How would you set up a CI/CD pipeline for this project?

A CI/CD pipeline (e.g., GitHub Actions) would: (1) run pytest tests/ -v on every pull request to catch regressions; (2) lint code with ruff or flake8; (3) optionally run a fast smoke-test training on a data sample; (4) on merge to main, build and push a Docker image containing the FastAPI service; (5) deploy to staging and run integration tests before promoting to production. MLflow model promotion can be integrated as a manual gate.

Section 4: Ethics & Regulation

Q36. What is the Equal Credit Opportunity Act (ECOA) and how does it affect your model?

The ECOA (US federal law) prohibits lenders from discriminating against credit applicants based on race, colour, religion, national origin, sex, marital status, or age. Even if these variables are not directly included in the model, proxy variables (e.g., zip code, which correlates with race) can produce discriminatory outcomes — known as disparate impact. Models must be tested for discriminatory effects regardless of intent.

Q37. How would you check your model for demographic bias?

Steps include: (1) compute approval rates and false negative rates (missed defaults approved) segmented by protected group (age, gender, race — inferred via proxy or reported where legal); (2) measure disparate impact ratio (approval rate of minority group / approval rate of majority group; a ratio < 0.8 triggers the '4/5ths rule'); (3) use algorithmic fairness metrics such as equalised odds or demographic parity; (4) run statistical tests to confirm any observed differences are not due to chance.

Q38. What is SHAP and how does it help with model explainability?

SHAP (SHapley Additive exPlanations) is a game-theory-based framework that assigns each feature a contribution score for a specific prediction. For a loan application predicted as high-risk, SHAP values show exactly how much each feature (e.g., DTI = +0.12, annual income = -0.08) pushed the prediction toward or away from default. This enables both global explanations (feature importance across all loans) and local explanations (why was this specific application denied).

Q39. How would you explain a loan denial decision to an applicant?

Adverse Action Notices are legally required in the US under ECOA and FCRA. The notice must state the principal reasons for denial in plain language — typically the top 3–4 SHAP factors. For example: 'Your application was declined primarily due to: (1) high debt-to-income ratio, (2) recent delinquency on your credit record, (3) short credit history.' The explanation must not reference protected characteristics.

Q40. What are the risks of using zip code or geographic data in a credit model?

Zip codes are a proxy for race and socioeconomic status due to historical residential segregation patterns. A model trained on zip codes may learn to deny credit to applicants in predominantly minority neighbourhoods even if it was never given race as an explicit feature — this is called redlining and is illegal under the Fair Housing Act and ECOA. If geographic data is used, the model must be audited for disparate impact and compared against a version without geographic features.
